

ELTE EÖTVÖS LORÁND UNIVERSITY

DATA INPUT AND VALIDATION

Modern web application development in .NET

Lecture 3

Dr. Máté Cserép

mcserep@inf.elte.hu

https://mcserep.web.elte.hu/

FORMS

Users might be requested to enter data on websites (e.g. during a simple login), which can be done with forms

- forms are built up from controls, whose content can be sent to the server in a **`POST`** type request
- forms can be created with the **`Html.BeginForm()`** operation
  - usually placed inside a **`@using`** block, this defines their scope;
  - inside the form we use **`input`** fields can be placed and **`submit`** type **`input`** element to send the request;
  - the **`value`** attribute of the input fields specifies which values (properties) of the model are inserted

FORMS

- e.g. (viewmodel):

```
public class UserData // viewmodel
{
  public int UserId { get; set; }
  public string UserName { get; set; }
  public string UserPass { get; set; }
  public string Birthdate { get; set; }
}
```

FORMS

- e.g. (view):

```
@model UserData
...
@using (Html.BeginForm()){ // begin of form
  <div>Your name:
    <input name="userName" value="@Model.UserName" />
    @* text input field for the UserName of the viewmodel*@
  </div>
  @* … further  input fields … *@
  <input type="submit" value="Login" />
  @* form submit button *@
} // end of form
```

DATA INPUT

Forms also run an action, but pass the data entered as a viewmodel

- They run the same action that created their view by default, but this can be parameterized, e.g.:
  `@using (Html.BeginForm("Index", "Login", …))`
- In the controller, we can specify that an action should be performed only on **`GET`** or **`POST`** requests (by adding the **`HttpGet`** or **`HttpPost`** attribute), thus the two operations can be separated
  - without attributes, the two HTTP methods must be handled in one operation (as overloading is not allowed in this case)

DATA INPUT

- e.g.:

```
public class LoginController : Controller {
  [HttpGet] // executed upon simple page load
  public ActionResult Index() {
    return View(); // return the view without viewmodel
  }

  [HttpPost] // executed upon form submission
  public ActionResult Index(UserData data) {
  // the form data is retrieved as a function parameter
  …
}
```

DATA INPUT

- In a form, input fields can also be generated using operations, e.g.: **`@Html.TextBox("userName", "@Model.UserName")`**
- With a strongly typed viewmodel, input fields can also be generated for a specific property of the viewmodel, e.g.: **`@Html.TextBoxFor(m => m.UserName)`**
- The following input fields can be generated this way:
  - TextBox, TextArea, Password
  - CheckBox, RadioButton, DropDownList, ListBox
  - Hidden

DATA INPUT

- e.g.:

```
@model UserData
...
@using (Html.BeginForm()){ // begin of form
  <div>Your name:
       @Html.TextBoxFor(m => m.UserName)</div>
  <div>Your password:
       @Html.PasswordFor(m => m.UserPass)</div>
  @* generate input fields for the properties *@
  ...
  <input type="submit" value="Login" />
} // end of form
```

DATA INPUT

When the type of model properties are not known in advance, dynamically generated elements can be used:

- the **`Editor`** operation dynamically generates the input field; while
- the **`Label`** operation generates the label; and
- the **`Display`** operation displays the content in read-only mode for the specified property.

When processing each property one by one is not required, the whole viewmodel can be displayed (**`LabelForModel`**, **`DisplayForModel`**) or the editable form can be generated (**`EditorForModel`**)

- in this case it is often useful to annotate the viewmodel for proper field generation

DATA INPUT

- e.g.:

```
@model UserData // viewmodel of user data
...
@using (Html.BeginForm()) { // begin of form
  <div>@Html.LabelFor(m => m.UserName):
       @Html.EditorFor(m => m.UserName)</div>
  @* dynamically generate label and input field *@
  ...
  <div>@Html.LabelFor(m => m.Birthdate):
       @Html.EditorFor(m => m.Birthdate)</div>
  @* date picker will be displayed here*@
  <input type="submit" value="Login" />
} // end of form
```

DATA INPUT

- e.g.:

```
@model UserData // viewmodel of user data
...
@using (Html.BeginForm()) { // begin of form
  @Html.EditorForModel()
  @* make the whole viewmodel editable *@

  <input type="submit" value="Login" />
  @* we still need a submit button though*@
} // end of form
```

VIEWMODELS FOR FORMS

Our viewmodel and its properties can be *annotated* with a number of attributes that affect the dynamic generation of the form fields

- this way we have better control how data is displayed or requested from the users at generation time, e.g.:
  - content of the label (**`Display`**)
  - format of the content (**`DisplayFormat`**)
  - hide, but include content in the form (**`HiddenInput`**)
  - further specify data type (**`DataType`**),
    or the type of the input field (**`UIHint`**)

VIEWMODELS FOR FORMS

- e.g.:

```
[DisplayName("User login:")] // set label for the VM
public class UserData
{
  // hidden, do not even render as hidden field
  [HiddenInput(DisplayValue=false)]
  public int UserId { get; set; }

  [Display(Name="Your name:")] // set label
  public string UserName { get; set; }
```

VIEWMODELS FOR FORMS

- e.g.:

```
    [Display(Name="Your password:")]
    [UIHint("Password")] // set to password input field
    public string UserPass { get; set; }

    [Display(Name="Your birthday:")]
    [DisplayFormat(DataStringFormat="yy.MM.dd")]
    [DataType(DataType.Date)] // display only the date
    public DateTime Birthdate { get; set; }
}
```

DISPLAY TEMPLATES

Both for display (**`DisplayFor`**) and input (**`EditorFor`**) fields, not only built-in elements, but custom partial views can also be used.

- these are called display templates and are placed in the **`Views/Shared/DisplayTemplate`** directory
- the type of the property will be the viewmodel of the partial view
- e.g.:

```
[Display(Name="Your name:")]
[UIHint("MyNameDisplay")]
// loads the MyNameDisplay.cshtml view
public string UserName { get; set; }
```

TAG HELPERS FOR FORMS

Tag helper syntax can also be used to dynamically generate forms in ASP.NET Core, resulting in more concise codebase that is easier to read:

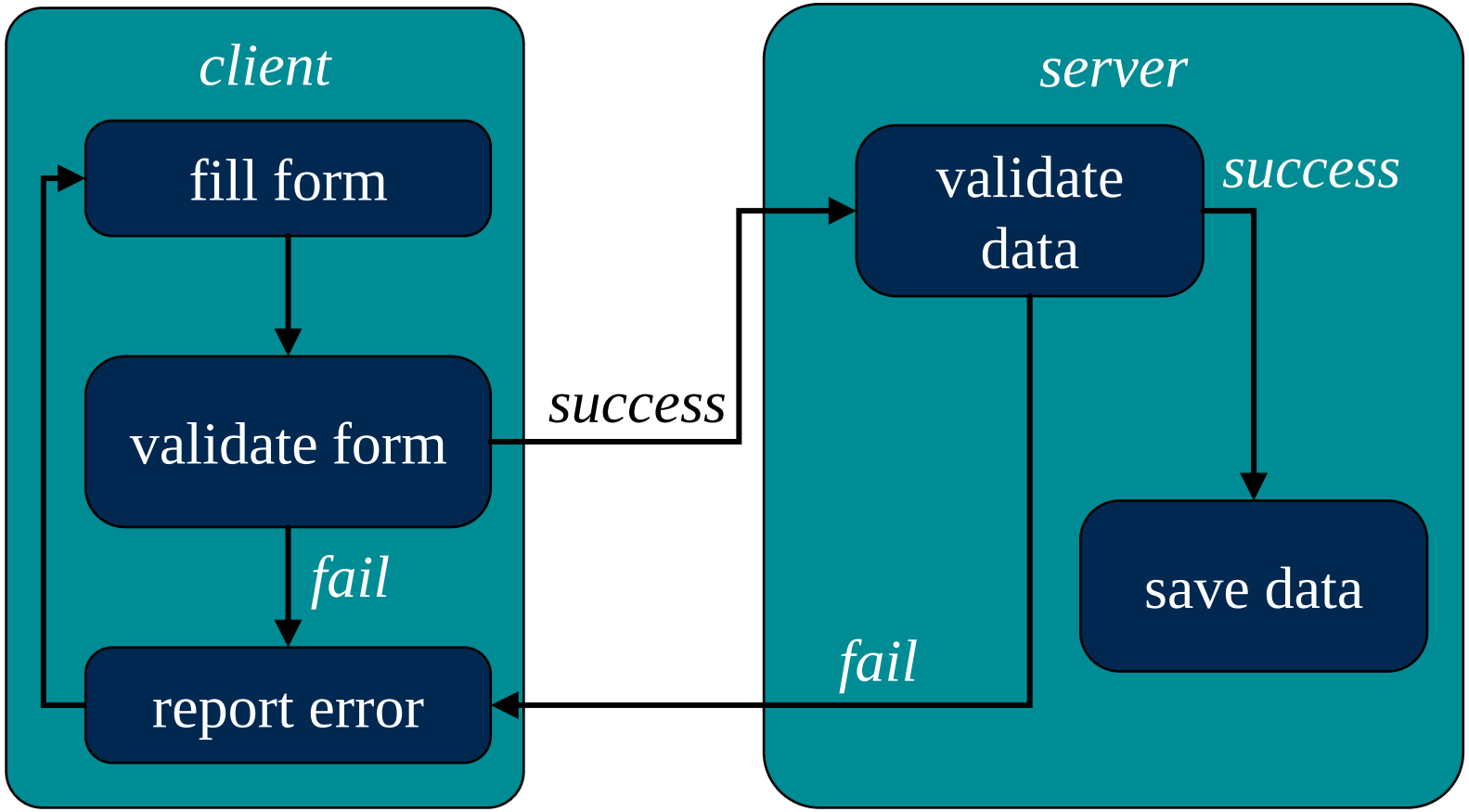
```
@* begin of form *@
<form asp-action="Results"> @* set the action URL *@
  <div><label asp-for="UserName"></label>
       <input asp-for="UserName" /></div>
  @* the label and the input is generated dynamically*@
  <div><label asp-for="UserPass"></label>
       <input asp-for="UserPass" /></div>
  <div><label asp-for="BirthDay"></label>
       <input asp-for="BirthDay" /></div>
</form>
```

VALIDATION

We always need to check the data we receive from the user, this is called the *validation of the (view)model*, which consists of the following steps:

1. On the client side, as soon as the user submits the form:
   - check that the necessary data have been provided;
   - check that the type and format are correct (e.g. e-mail, date);
   - inform the user if there are any problems.
2. When successful, the data will be sent to the server
3. The same validations are performed again on the server side, but this time including *security checks* as well
4. In case of error, report back to the client, otherwise the requested action be executed

VALIDATION


client
fill form
validate form
fail
report error
success
server
validate data
success
save data
fail

VALIDATION

Validation can be done on the server side only, or client and server side

- server-side validation is essential, especially to prevent attacks
- validation can be done completely manually or using built-in tools

On the server side, model state can be managed in the controller through the **`ModelState`** property

- the **`IsValid`** value specifies that all required values are found, has the correct type and passed the validation
- the **`AddModelError`** operation can be used to report an error

VALIDATION

- e.g.:

```
public class LoginController : Controller {
...
[HttpPost]
public IActionResult Index(UserData data) {
  if (string.IsNullOrEmpty(data.UserName))
    // name is empty
    ModelState.AddModelError(
      "UserName", "User name is required!");
    // report the error for the property
  …
  if (ModelState.IsValid)
     // when data passed validation
```

VALIDATION IN THE VIEW

Errors can be specified globally or for each property individually (in the former case, no property is specified)

In the view, you can display error messages

- for a property, the **`Html.ValidationMessageFor()`** method displays the error message indicated
- for the whole model, **`Html.ValidationSummary()`** method displays the error messages
    - it can be specified with a Boolean parameter whether all errors or only those for which no property is specified should be displayed (**`Html.ValidationSummary(true)`**)

VALIDATION IN THE VIEW

- e.g.:

```
@using (Html.BeginForm()){
  <div> @Html.ValidationSummary(true, "Errors:")
  @* global errors *@
  </div>
  <div>
    @Html.LabelFor(m => m.UserName):
    @Html.EditorFor(m => m.UserName)
    @Html.ValidationMessageFor(m => m.UserName)
    @* errors reported for the UseName property *@
  </div>
  ...
```

VALIDATION IN THE VIEW

- tag helpers can be used for this purpose as well:

```
<form>
  <div asp-validation-summary="ModelOnly">
 @* global errors *@
  </div>
  <div>
    <label asp-for="UserName"></label>
    <input asp-for="UserName" />
    <span asp-validation-for="UserName"></span>
    @* errors reported for the UseName property *@
  </div>
  ...
```

- The **`asp-validation-summary`** can be set to **`All`** or **`ModelOnly`**, in the former case it also prints the errors for each property.

VALIDATION IN THE VIEWMODEL

We can specify the validation criteria directly in the viewmodel, automating the validation

- the validation criteria can be defined per property and the attached error message (**`ErrorMessage`**) can also be specified
- some common validator:
  - **`Required`**
  - **`StringLength`**, **`RegularExpression`**, **`Range`**, **`Compare`**
  - **`Url`**, **`Phone`**, **`CreditCard`**, **`EmailAddress`**
- validation can be performed manually using the **`TryValidateObject`** method of the **`Validator`** class

VALIDATION IN THE VIEWMODEL

- e.g.:

```
public class UserData {
  ...
  [Required(ErrorMessage="User name is required.")]
  [StringLength(15,
    "User name cannot be longer than 15 characters.")]
  [RegularExpression("^[a-z0-9_-]{3,15}$",
    "User name contains invalid characters.")]
  // validation criteria for the UserName
  public string UserName { get; set; }
  …
}
```

CLIENT-SIDE VALIDATION

ASP.NET MVC supports client-side validation with JavaScript

- The easiest solution is to use the *jQuery Validation* library, which can automatically handle validation annotations defined in the server-side model and apply them on the client-side.

```
...
<script src="~/lib/jquery/dist/jquery.min.js">
<script src="…/jquery.validate.js">
<script src="…/jquery.validate.unobtrusive.js">
...
```

- The validation is done before the form is submitted, on the client side (the annotations are properly embedded in the HTML controls).

CLIENT-SIDE VALIDATION

- In *production* environments, it may be useful to load JavaScript libraries from a *CDN (Content Delivery Network)*, but as a secondary solution, you can also provide a local version of the file:

```
<script src="https://ajax.aspnetcdn.com/ajax/jquery/
             jquery-2.2.0.min.js“
        asp-fallback-src=
            "~/lib/jquery/dist/jquery.min.js“
        asp-fallback-test="window.jQuery“
        crossorigin="anonymous“
        integrity="{hash}">
</script>
```

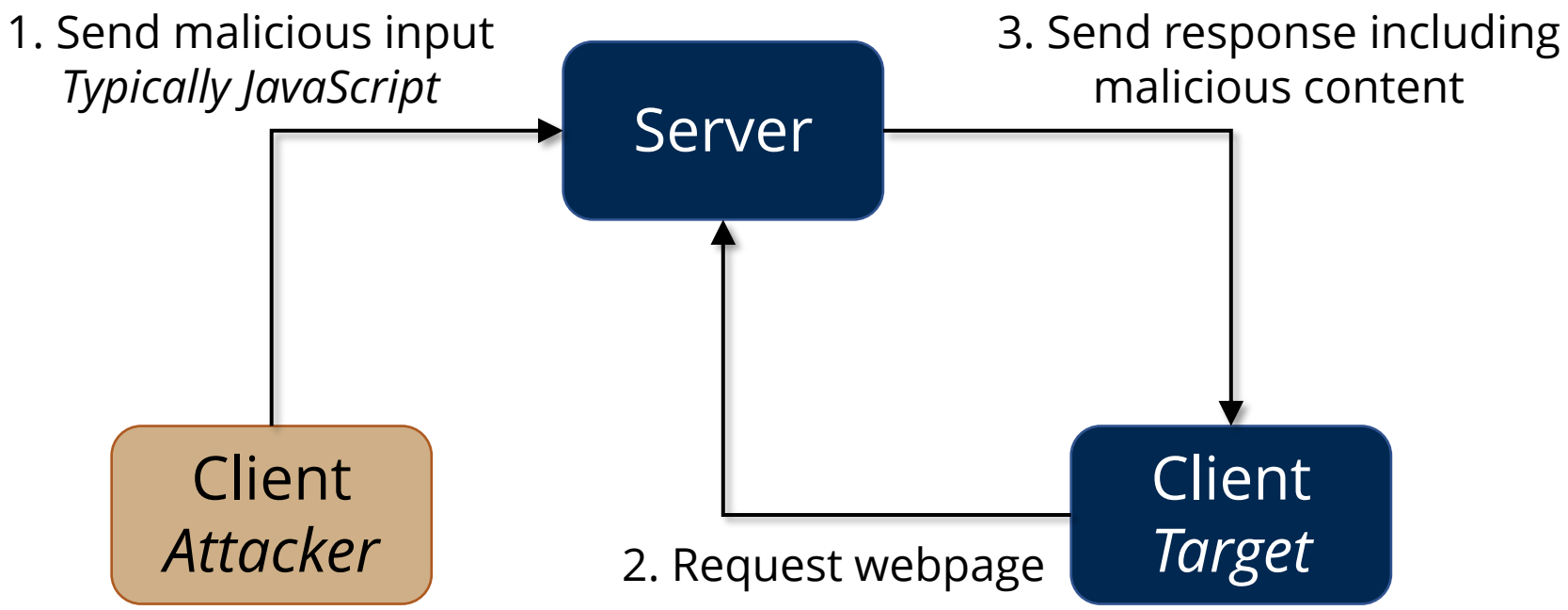
SECURITY VALIDATION

Data sent by users may contain malicious content, so validation is also crucial from a security point of view. Some of the most common attacks:

- *SQL injection:* manipulates SQL statements executed on the server
  - cannot occur when using an ORM approach like EF
- *cross-site scripting (XSS):* a client script is uploaded to the server and executed by the targeted client's browser
  - Razor encodes HTML content by default, but manual input sanitization could be required for other purposes
  - manual override is possible with the **`Html.Raw()`** and the **`Html.Encode()`** helper functions

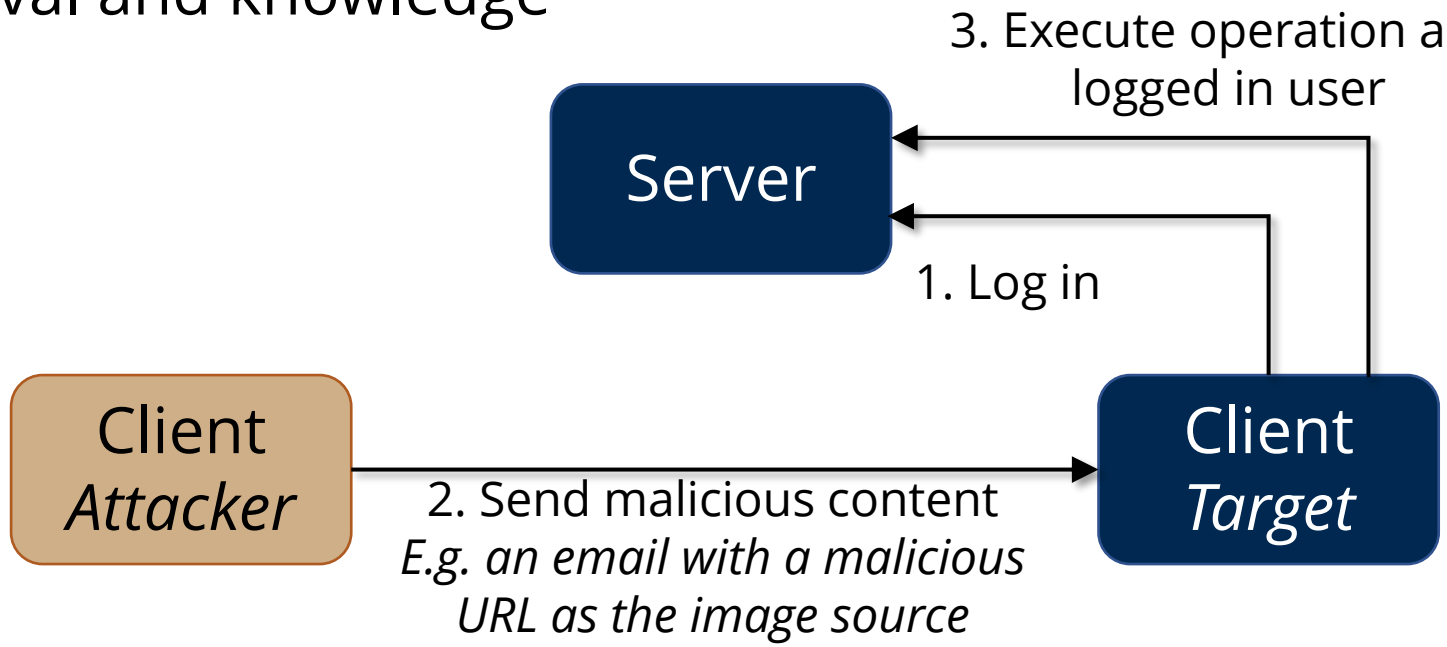
SECURITY VALIDATION

- demonstrating a *cross-site scripting (XSS)* attack:



SECURITY VALIDATION

- *cross-site request forgery (XSRF):* redirecting the user and executing a request without his/her knowledge
  - e.g. the user submits a form with content (**POST**) without his/her approval and knowledge



SECURITY VALIDATION

- How to defend against *cross-site request forgery (XSRF)* attacks on the server side?
  - an attack can be avoided by making sure that the same client filled and then sent the form
  - to achieve this, we can place a token (**`Html.AntiForgeryToken()`**) in the form, which is automatically sent back by the client
  - when the action processing the form submission is executed, we can retrieve and validate the token (manually or with the **`ValidateAntiForgeryToken`** attribute)
    - if the two values match, then there was no attack

SECURITY VALIDATION

- Using either the **`Html.BeginForm()`** helper method or tag helpers, inclusion of *an anti-forgery token* is the default for the POST method.
  - Generated HTML content:

```
<form method="post">
...
<input name="__RequestVerificationToken“
       type="hidden" value="{token}" />
</form>
```

  - Can be disabled if necessary, e.g. for tag helper:

```
<form method="post" asp-antiforgery="false">
```

  - Can be added manually to a form if necessary:

```
@Html.AntiForgeryToken()
```